

UNIVERSIDAD DE GUADALAJARA



CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS

INGENIERÍA EN COMPUTACIÓN

ARQUITECTURA DE COMPUTADORAS

MIPS

Fase 2

Nombres:

- Luis Giovanni Rodríguez Hernandez
- Sara Iriel Alonso Jaquez
- Leonel Octavio Rios Loreto
- Jorge Abraham Moran Encina

Fecha: 16 de Mayo de 2026

1. Introducción

En esta segunda fase del proyecto MIPS se amplía el DataPath desarrollado en la Fase 1 para soportar nuevas instrucciones de la arquitectura MIPS, principalmente instrucciones Tipo I y Tipo J. Mientras que en la primera etapa únicamente se ejecutaban operaciones aritméticas y lógicas entre registros, ahora el procesador puede acceder a memoria, realizar comparaciones para saltos condicionales y modificar el flujo de ejecución del programa.

El objetivo principal de esta fase es implementar un DataPath de ciclo único de 32 bits que permite ejecutar instrucciones como LW (Load Word), SW (Store Word), BEQ (Branch if Equal) y J (Jump), además de continuar soportando las instrucciones Tipo R que ya estaban implementadas.

También se trabajó en el proceso de decodificación de instrucciones ensamblador a código máquina binario, con el propósito de generar archivos de memoria capaces de ser cargados directamente en la memoria de instrucciones mediante archivos .mem o .txt.

En esta etapa también se comienza a introducir el concepto de segmentación o pipeline, agregando buffers entre etapas del procesador. Esto representa un acercamiento más realista a la arquitectura utilizada en procesadores modernos, permitiendo mejorar el rendimiento al ejecutar varias instrucciones simultáneamente en distintas etapas del procesamiento.

Formato de instrucción tipo R

Una instrucción MIPS tipo R tiene 32 bits de ancho y está dividida en 6 campos:

Campo	Bits	Tamaño	Descripción
OpCode	31:26	6 bits	Código de operación (siempre 000000 para tipo R)
RS	25:21	5 bits	Registro fuente 1
RT	20:16	5 bits	Registro fuente 2
RD	15:11	5 bits	Registro destino
Shamt	10:6	5 bits	Desplazamiento (no usado en estas instrucciones)
Function	5:0	6 bits	Código de función que especifica la operación

La separación de estos campos no es física en el hardware: la instrucción llega como 32 bits y cada módulo toma los bits que le corresponden según su posición.

2. Marco Teórico

Arquitectura MIPS y Arquitectura RISC

MIPS (Microprocessor without Interlocked Pipeline Stages) es una arquitectura RISC (Reduced Instruction Set Computer) diseñada para simplificar el hardware mediante instrucciones sencillas y de tamaño fijo. Todas las instrucciones tienen una longitud de 32 bits, lo que facilita la decodificación y ejecución dentro del procesador.

A diferencia de arquitecturas CISC como x86, donde las instrucciones tienen tamaños variables y operaciones complejas, MIPS utiliza instrucciones simples que normalmente realizan una sola operación. Esto permite que el diseño del procesador sea más eficiente y más fácil de implementar mediante hardware digital.

Instrucciones Tipo I

Las instrucciones Tipo I utilizan registros y un valor inmediato de 16 bits. Este tipo de instrucciones se emplea principalmente para acceso a memoria, operaciones inmediatas y saltos condicionales.

El formato de una instrucción Tipo I es el siguiente:

Campo	Bits	Tamaño	Descripción
Opcode	31:26	6 bits	Código de Operación
RS	25:21	5 bits	Registro fuente
RT	20:16	5 bits	Registro destino
Immediate	15:0	16 bits	Constante o desplazamiento

Instrucción LW (Load Word)

La instrucción LW permite cargar un dato almacenado en memoria hacia un registro del Banco de Registros, la dirección efectiva se calcula sumando el contenido del registro base RS con el valor inmediato extendido a 32 bits:

Dirección = $R_s + \text{offset}$

Posteriormente, el dato almacenado en memoria es leído y guardado en el registro RT.

Instrucción SW (Store Word)

La instrucción SW realiza el proceso contrario a LW. En lugar de leer memoria, almacena el contenido de un registro dentro de la memoria de datos.

La dirección también se calcula usando:

$$\text{Dirección} = \text{Rs} + \text{offset}$$

Instrucción BEQ (Branch if Equal)

La instrucción BEQ permite realizar saltos condicionales comparando dos registros. Si ambos registros son iguales, el procesador modifica el Program Counter (PC) para saltar a otra dirección del programa.

La comparación se realiza mediante una resta en la ALU. Si el resultado es cero, se activa la señal Zero y el salto se ejecuta.

La dirección del salto se calcula mediante:

$$\text{PC} = \text{PC} + 4 + (\text{Offset} \times 4)$$

Instrucciones Tipo R

Las instrucciones tipo R son las instrucciones aritméticas y lógicas de MIPS. Se llaman "tipo R" porque todos sus operandos provienen de Registros. Su OpCode siempre es 000000 (6 bits en cero), y la operación específica se distingue por el campo Function (bits 5:0).

Los códigos de función para las instrucciones implementadas son:

Instrucción	OpCode	Función (binario)	Función (hex)	Descripción
ADD	000000	100000	0x20	Suma con signo: $RD = RS + RT$
SUB	000000	100010	0x22	Resta con signo: $RD = RS - RT$
AND	000000	100100	0x24	AND bit a bit: $RD = RS \& RT$
OR	000000	100101	0x25	OR bit a bit: $RD = RS \mid RT$
SLT	000000	101010	0x2A	Set Less Than: $RD = 1$ si $RS < RT$
NOR	000000	100111	0x27	NOR bit a bit: $RD = \sim(RS \mid RT)$

3. Objetivos

Objetivo General

Desarrollar un DataPath MIPS de 32 bits que pueda ejecutar instrucciones Tipo R, Tipo I y Tipo J incorporando acceso a memoria y preparación para arquitectura pipeline mediante buffers.

Objetivos Específicos

- Agregar módulos para soportar instrucciones LW, SW y BEQ.
- Implementar la lógica necesaria para acceso a memoria.
- Diseñar la lógica de comparación y salto condicional.
- Incorporar buffers para preparación de pipeline.
- Modificar la Unidad de Control para soportar nuevas instrucciones.
- Validar el funcionamiento del sistema mediante simulaciones y TestBench.

4. Desarrollo

El diseño sigue la arquitectura clásica de Patterson & Hennessy (single-cycle datapath) y está implementado en Verilog sintetizable estándar, sin constructos de SystemVerilog.

Instrucciones Tipo R

Se identifican por opcode 000000. La operación específica se determina por el campo func de 6 bits. El resultado siempre se escribe en el registro RD.

Instrucción	Opcode	func	Resultado	Descripción
add \$rd,\$rs,\$rt	000000	100000	$rd = rs + rt$	Suma entera de dos registros
sub \$rd,\$rs,\$rt	000000	100010	$rd = rs - rt$	Resta entera de dos registros
and \$rd,\$rs,\$rt	000000	100100	$rd = rs \& rt$	AND lógico bit a bit
or \$rd,\$rs,\$rt	000000	100101	$rd = rs rt$	OR lógico bit a bit
slt \$rd,\$rs,\$rt	000000	101010	$rd = (rs < rt) ? 1 : 0$	Comparación con signo
nor \$rd,\$rs,\$rt	000000	100111	$rd = \sim(rs rt)$	NOR lógico bit a bit

Instrucciones Tipo I de Memoria

Utilizan el campo inmediato de 16 bits con extensión de signo para calcular la dirección de memoria. El registro destino es siempre RT.

Instrucción	Opcode	Ext. Inmediato	Descripción
lw \$rt, imm(\$rs)	100011	Sign-extend	$rt = Mem[rs + sign_ext(imm)]$ — Carga una palabra de memoria de datos al registro
sw \$rt, imm(\$rs)	101011	Sign-extend	$Mem[rs + sign_ext(imm)] = rt$ — Guarda el valor de un registro en memoria de datos

Instrucciones Tipo I Nuevas — Fase 2

El campo inmediato de 16 bits se extiende a 32 bits antes de ingresar a la ALU. La forma de extensión depende de la instrucción: las aritméticas usan sign-extend y las lógicas usan zero-extend. El registro destino es siempre RT.

Instrucción	Opcode	Ext. Inmediato	Resultado	Descripción
addi \$rt,\$rs,imm	001000	Sign-extend	$rt = rs + \text{sign}(imm)$	Suma con inmediato. Permite constantes negativas.
slti \$rt,\$rs,imm	001010	Sign-extend	$rt = (rs < imm) ? 1 : 0$	Compara registro con constante con signo.
ori \$rt,\$rs,imm	001101	Zero-extend	$rt = rs \text{zero}(imm)$	OR con patron de bits. Zero-extend evita alterar bits superiores.
andi \$rt,\$rs,imm	001100	Zero-extend	$rt = rs \& \text{zero}(imm)$	AND con mascara de bits. Zero-extend evita alterar bits superiores.

Por que la diferencia en la extensión: addi y slti trabajan con valores numericos que pueden ser negativos (por ejemplo, restar 5 equivale a sumar -5). Si se usara zero-extend, el valor -5 (0xFFFFB en 16 bits) se interpretaría como 65531, resultado incorrecto. ori y andi operan sobre patrones de bits. Si se extendiera con el bit de signo, los bits superiores del inmediato quedarían en 1, alterando el resultado de la operación lógica.

2.4 Instrucciones Nuevas — Branch y Jump

Instrucción	Opcode	Tipo MIPS	Cálculo del destino
beq \$rs,\$rt,offset	000100	Tipo I	Si $rs == rt$: $PC = PC + 4 + \text{sign_ext}(\text{offset}) \ll 2$. La ALU calcula $rs - rt$; si el resultado es cero se activa el flag Zero y el salto ocurre.
j target	000010	Tipo J	$PC = \{ PC + 4[31:28], \text{inst}[25:0], 2'b00 \}$. Los 26 bits del campo target se desplazan 2 posiciones y se combinan con los 4 bits superiores del $PC + 4$.

Módulos modificados

PC.v — Program Counter

Corrección aplicada: la asignación bloqueante (=) dentro del bloque always @(posedge CLK) fue reemplazada por no-bloqueante (<=). En Verilog, usar asignación bloqueante en lógica síncrona produce comportamiento ambiguo en simulación y puede generar circuitos incorrectos en síntesis. El reset activo-alto fue preservado.

ALUMIPS.v — ALU de 32 bits

Se agregó la salida Zero, un wire combinacional que vale 1 cuando el resultado de la ALU es exactamente cero. Esta señal es indispensable para que beq pueda detectar la igualdad entre dos registros. Adicionalmente, la operación slt fue corregida para usar comparación con signo mediante \$signed(), necesario para que slti funcione correctamente con valores negativos.

ALUCTRL.v — Unidad de Control de la ALU

Se ampliaron los casos de decodificación con nuevos valores de AluOp para cubrir cada instrucción Tipo I nueva. La lógica de R-type que usa el campo func se preserva sin modificaciones.

AluOp	Instrucción origen	SelAlu resultante	Operación ALU ejecutada
000	lw / sw	0010	ADD — calculo de dirección de memoria
001	beq	0110	SUB — genera el flag Zero cuando rs == rt
010	R-type (via func)	variable	Según el campo func[5:0] de la instrucción
011	addi	0010	ADD — suma registro más inmediato extendido
100	slti	0111	SLT con signo — compara registro con inmediato
101	ori	0001	OR — operación lógica con inmediato extendido a cero
110	andi	0000	AND — mascara lógica con inmediato extendido a cero

UNITCTRL.v — Unidad de Control Principal

Es el módulo con mayor número de cambios. Se agregaron tres señales de salida nuevas y seis nuevos casos en el decodificador de opcode.

- Branch: activa la lógica de salto condicional. Solo se activa con beq.
- Jump: activa la lógica de salto incondicional. Solo se activa con j.
- ImmSel: selecciona el tipo de extensión del inmediato. Valor 0 aplica sign-extend (addi, slti, lw, sw, beq). Valor 1 aplica zero-extend (ori, andi).

Módulos nuevos

ImmExtend.v — Extensor de Inmediato

Reemplaza al módulo Sign-extend.v de la Fase 1. Recibe el campo inmediato de 16 bits y la señal ImmSel de la unidad de control.

- ImmSel = 0 (sign-extend): replica el bit 15 en los 16 bits superiores. Usado por addi, slti, lw, sw y beq.
- ImmSel = 1 (zero-extend): rellena los 16 bits superiores con cero. Usado por ori y andi.

ShiftLeft2.v — Desplazamiento izquierda 2 bits

Desplaza el inmediato extendido 2 posiciones a la izquierda para calcular el offset del branch target. Este desplazamiento es necesario porque el offset en el campo inmediato de beq esta expresado en palabras (unidades de 4 bytes), y el PC trabaja en bytes. Multiplicar por 4 equivale a desplazar 2 bits.

Mux4a1.v — Multiplexor para selección del siguiente PC

Selecciona entre tres posibles fuentes para el siguiente valor del PC. La logica de seleccion dentro de DPTR establece prioridad: Jump tiene prioridad sobre Branch, y ambos tienen prioridad sobre la ejecucion secuencial.

PCsel	Fuente del PC	Condicion de activacion
00	PC + 4	Ejecucion secuencial. Ningun salto activo.
01	Branch Target	beq activo (Branch=1) Y flag Zero activo (resultado ALU = 0).
10	Jump Target	Instruccion j activa (Jump=1). Siempre toma el salto incondicional.

TestBench – TB_DPTR.v

El módulo TB_DPTR fue desarrollado para verificar el funcionamiento del datapath MIPS implementado en la Fase 2. Este TestBench se encarga de generar las señales necesarias para la simulación, incluyendo el reloj (CLK) y la señal de reinicio (RESET), permitiendo ejecutar instrucciones reales almacenadas en el archivo inst.txt. Instancia el módulo principal DPTR, el cual contiene todos los componentes del datapath, como el Program Counter, la memoria de instrucciones, la ALU, la unidad de control y el banco de registros.

```
`timescale 1ns/1ns
module TB_DPTR;
    reg CLK;
    reg RESET;
    DPTR mydptr(.CLK(CLK), .RESET(RESET));
    initial CLK = 1'b0;
    always #10 CLK = ~CLK;
    initial begin
        RESET = 1'b1;
        #25;
        RESET = 1'b0;
    end
    initial begin
        $display("=== Testbench DPTR Fase 2 ===");
        $display("Tiempo | PC          | Instruccion | Reg destino | Resultado
ALU");
        $monitor("%0t      | %h | %h | R%-2d=%0d",
            $time,
            mydptr.DIR,
            mydptr.InstTR,
            mydptr.AW,
            mydptr.ALUResult);
    end
    initial begin
        #500;
        $display("=== Fin de simulacion ===");
        $finish;
    end
    initial begin
        $dumpfile("tb_dptra.vcd");
        $dumpvars(0, TB_DPTR);
    end
end
endmodule
```

Durante la simulación se muestran en consola distintos valores importantes del datapath, como el tiempo de simulación, el valor actual del Program Counter (PC), la instrucción ejecutada, el registro destino, el resultado obtenido por la ALU. Esto se realiza mediante las instrucciones \$display y \$monitor.

Tabla de Instrucciones de prueba

Programa de Prueba(inst.txt)

El archivo inst.txt contiene el programa de verificación en formato binario (1 byte por linea, big-endian). Prueba cada instrucción nueva y verifica que las instrucciones Tipo R sigan funcionando.

Estado inicial del Banco de Registros (heredado de Fase 1): R0=10, R1=11, R2=12, R3=13, R4=14, R5=15.

#	Instrucción	PC (hex)	Resultado esperado	Que verifica
1	addi \$8,\$0,100	0x00	R8 = 110	addi con inmediato positivo, sign-extend correcto
2	addi \$9,\$1,5	0x04	R9 = 16	addi desde un registro distinto de \$0
3	slti \$10,\$0,11	0x08	R10 = 1	slti verdadero: 10 es menor que 11
4	slti \$11,\$1,11	0x0C	R11 = 0	slti falso: 11 no es estrictamente menor que 11
5	ori \$12,\$0,15	0x10	R12 = 15	ori con zero-extend: 10 OR 15 = 15
6	andi \$13,\$1,14	0x14	R13 = 10	andi con zero-extend: 11 AND 14 = 10
7	add \$14,\$0,\$1	0x18	R14 = 21	Tipo R sigue funcionando correctamente
8	beq \$0,\$0,+1	0x1C	PC = 0x24	beq tomado: \$0 == \$0, salta la instruccion 9
9	add \$15,\$2,\$3	0x20	SALTADA	Confirma que el branch funciono y salto esta instruccion
10	j 10	0x24	PC = 0x28	Jump incondicional a instruccion 10
11	j 10	0x28	PC = 0x28	Loop infinito, estado estable del simulador

5. Conclusión

La implementación de la Fase 2 permitió ampliar el funcionamiento del DataPath MIPS. Mientras que en la primera fase únicamente se ejecutaban operaciones entre registros, ahora el procesador es capaz de acceder memoria, realizar saltos condicionales y modificar el flujo de ejecución del programa.

Uno de los aspectos más importantes fue comprender cómo las instrucciones Tipo I requieren módulos adicionales como extensión de signo, desplazadores y lógica de branch. Esto permitió entender que el acceso a memoria y los saltos representan una mayor complejidad dentro del diseño de procesadores.

La implementación de buffers también ayudó a comprender cómo los procesadores modernos logran ejecutar varias instrucciones simultáneamente, mejorando el rendimiento general del sistema.

Finalmente, las simulaciones realizadas demostraron que el DataPath funciona correctamente para las instrucciones implementadas, validando tanto la lógica de control como el flujo de datos entre los diferentes módulos del procesador.

6. Referencias

Patterson, D. A., & Hennessy, J. L. (2014). Computer Organization and Design: The Hardware/Software Interface (5.^a ed.). Morgan Kaufmann Publishers.

Ciletti, M. D. (2011). Advanced Digital Design with the Verilog HDL (2.^a ed.). Pearson/Prentice Hall.

MIPS Technologies. (2005). MIPS32 Architecture for Programmers, Volume II: The MIPS32 Instruction Set. MIPS Technologies Inc.

Harris, D. M., & Harris, S. L. (2012). Digital Design and Computer Architecture (2.^a ed.). Morgan Kaufmann.